

PRÁCTICA 9

B A S E S D E D A T O S D I S T R I B U I D O S

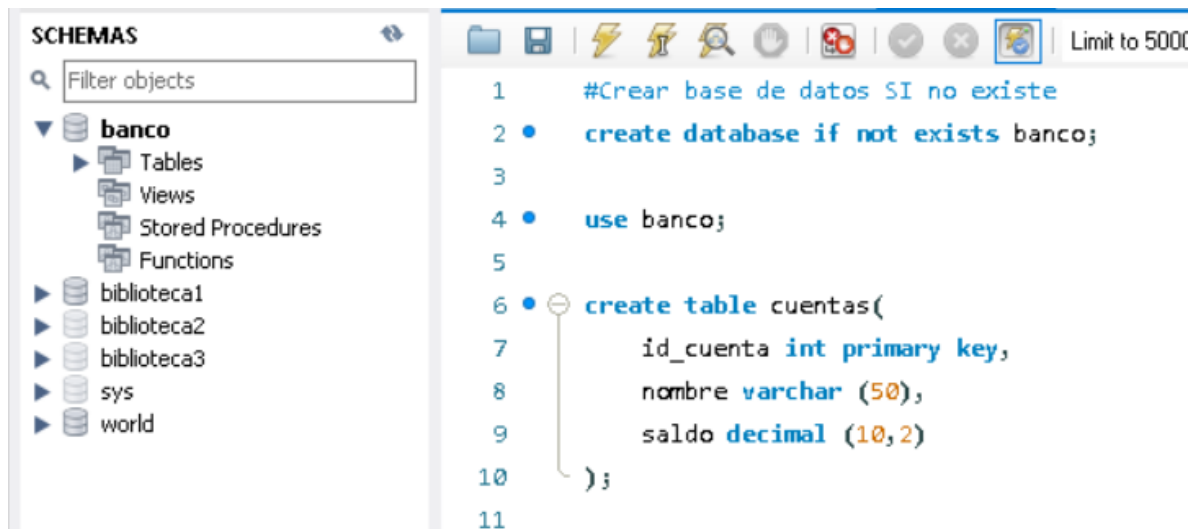
Gerardo Jimenez Haro

Jiovanni Lopez
220953241
6to semestre

28 de abril del 2026

Empezamos creando la base de datos banco con la condición de que la cree si es que no existe, si este ya existe devolverá una advertencia de que ya esta creada y no lo creara.

También creamos una tabla con las columnas id que será la llave primaria, nombre y saldo.



Con el uso de show tables verificamos que la tabla haya sido creada correctamente



Insertamos datos dentro de la tabla

```
--
16 #Insertar datos iniciales
17 • insert into cuentas values (1, 'Maria', 1000.00), (2, 'Juan', 800.00);
18
19 • select * from cuentas;
20
21 #Inicializar transaccion
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content

	id_cuenta	nombre	saldo
▶	1	Maria	1000.00
	2	Juan	800.00
	3	Pedro	1500.00
*	NULL	NULL	NULL

Inicializamos la transacción con start transaction.

Establecemos el parámetro AUTOCOMMIT a 0 para evitar que las transacciones hechas no se reflejen aun en la base de datos

```
17
20 #Inicializar transaccion
21 • start transaction;
22 #Iniciar autoccommit en 0
23 • set AUTOCOMMIT=0;
24 #verificar estado de autoccommit
25 • select @@autoccommit;
26
27 • insert into cuentas values (3, 'Pedro', 1500.00);
28
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	@@autoccommit
▶	0

Hacemos una transacción, en este caso, quitaremos 100 al saldo de María y se los agregaremos a Juan. Si verificamos los datos de la tabla desde workbench podemos ver que la transacción se hizo, pero si revisamos directamente desde donde esta alojada la base de datos podemos ver que la transacción no se ha reflejado en este.

```
29 • update cuentas set saldo = saldo - 100 where id_cuenta = 1;
30 • update cuentas set saldo = saldo + 100 where id_cuenta = 2;
31
32 • select * from cuentas;
33
34 #se confirma transaccion
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell C

	id_cuenta	nombre	saldo
▶	1	Maria	900.00
	2	Juan	900.00
	3	Pedro	1500.00
*	NULL	NULL	NULL

```
mysql> select * from cuentas;
+-----+-----+-----+
| id_cuenta | nombre | saldo |
+-----+-----+-----+
|          1 | Maria  | 1000.00 |
|          2 | Juan   |  800.00 |
|          3 | Pedro  | 1500.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

Podemos revertir el movimiento usando rollback. Podemos ver en la imagen que las cuentas regresaron a su estado original

```
--
32 • select * from cuentas;
33
34 #se confirma transaccion
35 • commit;
36 #revierte transaccion
37 • rollback;
```

Result Grid

	id_cuenta	nombre	saldo
▶	1	Maria	1000.00
	2	Juan	800.00
	3	Pedro	1500.00
•	NULL	NULL	NULL

Volvemos a ejecutar las líneas que harán una transacción entre cuentas y esta vez usaremos COMMIT para confirmar el cambio

```
34 #se confirma transaccion
35 • commit;
36 #revierte transaccion
37 • rollback;
```

Output

Action Output

#	Time	Action
✓ 28	09:59:24	select * from cuentas LIMIT 0, 5000
✓ 29	09:59:53	rollback
✓ 30	09:59:57	select * from cuentas LIMIT 0, 5000
✓ 31	10:00:19	update cuentas set saldo = saldo - 100 where id_cuenta = 1
✓ 32	10:00:19	update cuentas set saldo = saldo + 100 where id_cuenta = 2
✓ 33	10:00:31	commit

Después de usar COMMIT podemos ver que los datos se han actualizado tanto en workbench como en donde se está alojado la base de datos.

32 • `select * from cuentas;`

33

34 `#se confirma transaccion`

Result Grid

Filter Rows:

	id_cuenta	nombre	saldo
▶	1	Maria	900.00
	2	Juan	900.00
	3	Pedro	1500.00
*	NULL	NULL	NULL

cuentas 17

```
|      2 | Juan   | 900.00 |
+-----+-----+-----+
|      3 | Pedro  | 1500.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select * from cuentas;
+-----+-----+-----+
| id_cuenta | nombre | saldo |
+-----+-----+-----+
|      1 | Maria  | 900.00 |
|      2 | Juan   | 900.00 |
|      3 | Pedro  | 1500.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```